
Clase 164 — TD Learning, Q-Learning, Deep Q-Networks

Parte: 2 — Deep Learning · Fuente: Géron, cap. 18 § Q-Learning y § Deep Q-Learning.

Duración estimada: 80 min.

Clase 164 — TD Learning, Q-Learning, Deep Q-Networks

Parte: 2 — Deep Learning · Fuente: Géron, cap. 18 § Q-Learning y § Deep Q-Learning. Duración estimada: 80 min.

Objetivo

Implementar Q-Learning clásico (Watkins 1989) y su versión moderna DQN (Mnih et al. 2015, Nature paper que aprendió Atari desde pixels). Off-policy, model-free, bootstrap. Conocer los 2 trucos que hicieron a DQN funcionar: experience replay y target network.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

- Explicar la update Q-learning: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$.
- Implementar Q-learning tabular en FrozenLake.
- Construir un DQN: red state $\rightarrow Q(a)$ para todas las actions, MSE entre Q predicted y $r + \gamma \max_{a'} Q(s',a')$.
- Implementar replay buffer (almacenar transitions, samplear batch para training).
- Implementar target network (copia frozen actualizada cada N steps).

Temas

- TD (Temporal Difference) error: $\delta = r + \gamma V(s') - V(s)$.
- Q-learning: off-policy, sigue greedy de Q^* .
- ϵ -greedy exploration: con prob ϵ explora random, sino greedy.
- Replay buffer: deque de (s, a, r, s', done).
- Target network: estabilidad (sino, target se mueve mientras estimás).
- DQN sobre CartPole y Atari.

Definiciones y características

- Q-value $Q(s, a)$: expected return tras tomar a desde s.
- Bootstrap: update usando estimación actual (sin esperar fin de episodio).
- Off-policy: aprende sobre policy óptima aunque exploración sea ϵ -greedy.
- Replay buffer: almacena experiencias para sampleo iid.
- Target network Q' : copia de Q usada para calcular el target. Actualizada cada N steps.
- ϵ -greedy: balance exploration vs exploitation.

Dataset / recursos

- FrozenLake (tabular).
- CartPole (DQN).
- Librerías: gymnasium, tensorflow, keras, numpy.

Ejercicios

1. Q-learning tabular: en FrozenLake, mantener $Q[s, a]$ numpy array. Update con ϵ -greedy. Reportar success rate tras 5000 episodios.
2. DQN básico: red Dense(64) $\rightarrow$ Dense(64) $\rightarrow$ Dense(2) para CartPole. Sin replay buffer ni target network $\rightarrow$ ver inestabilidad.
3. Replay buffer: from collections import deque; buffer = deque(maxlen=10_000). Sample batch=32 random.
4. Target network: copiar $Q.weights$ cada 100 steps a Q_target .
5. ϵ decay: empezar $\epsilon=1.0$, decaer linealmente a 0.01 en 10 000 steps.

Homework verifiable

DQN sobre CartPole-v1:

1. Red Dense(64, relu) $\rightarrow$ Dense(64, relu) $\rightarrow$ Dense(2).
2. Replay buffer 50 000, batch 64.
3. Target network sync cada 100 steps.
4. ϵ : linear decay 1.0 $\rightarrow$ 0.05 sobre 10 000 steps.
5. Train hasta mean_reward(100 episodios) ≥ 195 .

Criterio de aceptación: alcanza ≥ 195 en ≤ 500 episodios; gráfico de return promedio muestra convergencia.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Q values explotan	Sin target network, bootstrap inestable. F
Convergencia lenta	Replay buffer muy chico o batch muy chico.
No explora $\rightarrow$ policy subóptima	ϵ muy chico desde el inicio. Fix: $\epsilon=1.0$ al
done confundido con truncated	Gymnasium 5-tuple. Fix: tratar terminated,
OOM con Atari	Stacks de 4 frames + buffer enorme. Fix: r

Preguntas frecuentes

¿DQN supera Q-learning tabular cuándo?

Cuando state space es enorme (continuous o pixels). Para FrozenLake (16 states), tabular gana.

¿Variantes de DQN?

- Double DQN (van Hasselt 2016): reduce overestimación.
- Dueling DQN (Wang 2016): separa V y advantage.
- Prioritized Experience Replay: muestreo no uniforme.
- Rainbow (Hessel 2018): combina todas.

¿DQN vs Policy Gradient?

DQN: off-policy, sample-efficient, action space discreto. PG (PPO): on-policy, más estable, action space continuo. Ambos en el zoo moderno.

¿Por qué replay buffer rompe la correlación temporal?

Sin él, transitions consecutivas están correlacionadas → batches no iid → gradiente sesgado.

¿Cuál env empezar?

CartPole para entender. LunarLander para algo más complejo. Atari para serio (requiere días).

Referencias

- Géron, cap. 18 — Q-Learning y Deep Q-Learning.
- Watkins (1989), Learning from Delayed Rewards (PhD thesis).
- Mnih et al. (2015), Human-level control through deep reinforcement learning, Nature — DQN paper.
- van Hasselt et al. (2016), Deep Reinforcement Learning with Double Q-learning.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — abrilo desde el laboratorio del programa o desde Jupyter.

Siguiente clase

Clase 165 — RL moderno: A3C, PPO, SAC (vista general)

Apéndice: notebook (primer bloque)

` 0(S) 1 2 3 4 5(H) 6 7(H)

```
import numpy as np
np.random.seed(42)

nS, nA = 16, 4
holes = {5, 7, 11, 12}
goal = 15

def next_state(s, a):
    row, col = divmod(s, 4)
    if a == 0: col = max(col - 1, 0)
    elif a == 1: row = min(row + 1, 3)
    elif a == 2: col = min(col + 1, 3)
    elif a == 3: row = max(row - 1, 0)
    return row * 4 + col

class GridWorld:
    def reset(self):
        self.s = 0
        return self.s
    def step(self, a):
        s2 = next_state(self.s, a)
        self.s = s2
        if s2 == goal: return s2, 1.0, True
        if s2 in holes: return s2, -1.0, True
        return s2, 0.0, False
```

```
env = GridWorld()  
print("GridWorld listo: 16 estados, 4 acciones, meta=15")
```

Archivos complementarios

- notebook.ipynb